TECHNICAL TEST · TAKE-HOME ASSIGNMENT

Software Engineer (Full-Stack) — 1-Day Practical Test

Build: StockFlow — Inventory & Sales (mini ERP module)

> **Stack:** React + TypeScript (frontend) · Go + PostgreSQL (backend) · the frontend consumes the REAL backend API
>
> **Deadline:** 1 calendar day — if you receive this test **today**, your submission is due **tomorrow before 23:59 (GMT+7)**. Expected focused effort: **6–8 hours maximum**. We evaluate end-to-end judgment, not overtime. If you run out of time, submit what works and tell us what you would do next.
>
> **Allowed:** Internet, official docs, any libraries, your own boilerplate.
>
> **Not allowed:** AI code generation (ChatGPT/Copilot/etc.), copying from other people or known open-source repos, mock/stub data in the final deliverable (the frontend must call the real API).

The Product

A warehouse supervisor uses one small app to manage **products**, adjust **stock**, and record **sales orders** that consume stock. You build the whole vertical slice: database → Go API → React UI.

This is a software-engineering test, not a styling exercise: the API contract, cross-stack error handling, and data safety under concurrency weigh more than visual polish.

Repository Layout (required)

```
stockflow/
├── backend/            # Go service (cmd/, internal/...)
├── frontend/           # React + TypeScript (Vite)
├── docker-compose.yml  # at minimum: postgres with migrations + seed applied
└── README.md
```

Part 1 — Backend: Go + PostgreSQL

Data model (starting point — extend if needed)

```
CREATE TABLE products (
    id         SERIAL PRIMARY KEY,
    sku        TEXT NOT NULL UNIQUE,
    name       TEXT NOT NULL,
    price      INTEGER NOT NULL CHECK (price >= 0),   -- IDR, integer: no float money
    stock      INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
    is_active  BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE stock_movements (
```

```
    id         SERIAL PRIMARY KEY,
    product_id INTEGER NOT NULL REFERENCES products(id),
    type       TEXT NOT NULL CHECK (type IN ('in', 'out')),
    qty        INTEGER NOT NULL CHECK (qty > 0),
    note       TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE sales_orders (
    id           SERIAL PRIMARY KEY,
    customer     TEXT NOT NULL,
    status       TEXT NOT NULL DEFAULT 'CREATED' CHECK (status IN ('CREATED','CANCELLED')),
    total        INTEGER NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    cancelled_at TIMESTAMPTZ
);

CREATE TABLE sales_order_items (
    id         SERIAL PRIMARY KEY,
    order_id   INTEGER NOT NULL REFERENCES sales_orders(id),
    sku        TEXT NOT NULL REFERENCES products(sku),
    qty        INTEGER NOT NULL CHECK (qty > 0),
    unit_price INTEGER NOT NULL
);
```

API contract (the frontend depends on exactly this)

Method	Path	Notes
GET	`/api/products?search=&page=&limit=`	`{data, page, limit, total}`; search matches sku or name, case-insensitive; each product includes `last_movement_at` (ISO string or `null`) — derive it from the movements table or maintain it on the row; document your choice
POST	`/api/products`	`201`; validate: sku required+unique, name required, price ≥ 0 integer
PUT	`/api/products/{id}`	`404` when missing
DELETE	`/api/products/{id}`	`204`; `409 {"error":{"code":"PRODUCT_HAS_HISTORY"}}` when the product has movement history — history is never deleted
POST	`/api/products/{id}/stock`	Body `{"type":"in"|"out","qty":n,"note":"..."}`; `409 {"error":{"code":"INSUFFICIENT_STOCK","available":n}}` when overdraw; movement row written in the same transaction
GET	`/api/products/{id}/movements`	History, newest first; `{"data": [...]}`
POST	`/api/orders`	Body `{"customer":"...","items":[{"sku":"...","qty":n}]}` — see rules below; returns `201` with the created order (items with unit prices + server-computed total)
POST	`/api/orders/{id}/cancel`	Restores stock; `409` when already cancelled
GET	`/api/orders/{id}`	Order + items + status + total

Order creation rules (the core of the test)

1. Validate everything before writing: unknown sku, `qty <= 0`, duplicate skus in one order, empty items → `400` with clear messages.
2. `unit_price` comes from the products table — the client never supplies prices. `total` is computed server-side.
3. Stock decrement for ALL items happens **atomically in one transaction**: either the whole order exists or nothing changed.
4. If any item is insufficient → `409` `{"error":{"code":"INSUFFICIENT_STOCK","sku":"...","available":n}}` and **no stock may have changed**.
5. Under concurrent orders for the same SKU, stock may never go negative. Guard inside the UPDATE (or row locks) — never SELECT-then-UPDATE.

Part 2 — Frontend: React + TypeScript

Screens & flows (must-have, in priority order)

1. **Products page** — table (SKU, Name, Price `Rp 1.234.567`, Stock, Status badge, Last movement "2h ago", Actions). Search (debounced) + filter: All / Active / Inactive / Low stock (≤ 10). Loading, empty, and error states with retry — all three must be reachable.
2. **Adjust stock modal** — direction, qty, note. Validation (integer > 0), submit disabled while saving, and the backend's `409 INSUFFICIENT_STOCK` must appear in the UI **with the available quantity**, never only in the console.
3. **Create order flow** — pick customer name + one or more items (sku, qty); on success show the order with its server-computed total; on `409` highlight exactly which SKU failed and what was available; on `400` show the validation messages.
4. **Order detail** — items, status, total; **Cancel** with confirmation; after cancelling, stock is visibly restored in the products list.
5. **Summary cards** — total products, active products, total stock value, low-stock count — derived from server data and refreshed after mutations.

Frontend technical requirements

- Vite (`npm create vite@latest -- --template react-ts`); TypeScript **strict**, shared API types in one module, no `any` on domain data.
- One API client module owns all `fetch` calls and error envelope parsing; components never build URLs ad hoc.
- Dev setup must work with the backend running (Vite proxy or CORS middleware — your call, document it).

Part 3 — Engineering Practices (weighted as much as the features)

1. **Runnable in ≤ 5 minutes**: `docker compose up` brings up Postgres with migrations + seeds applied (all apps in compose is a bonus; otherwise document the two commands clearly).
2. **Tests**: backend — at least (a) insufficient stock rejected with NOTHING written and (b) happy path writes movement + updates stock; a concurrency test (`go test -race`) is a strong bonus. Frontend — unit tests for validation/pure logic (Vitest).
3. **Seed data**: 12+ realistic products, a couple of sample orders.
4. **Git**: meaningful commits across the day on both sides — not one final dump.
5. **README**: how to run (step by step), curl examples for the API, design decisions, what you would improve, honest hours spent.

Bonus (only if must-haves are done)

- `Idempotency-Key` support on `POST /api/orders` (retries never double-order).
- docker compose runs db + backend + frontend end to end.
- `GET /api/reports/daily-sales?date=` + a summary card.
- Skeleton loading rows, `useDebounce` hook, tasteful styling.

How You Will Be Evaluated (summary)

Data safety under concurrency first, then cross-stack correctness (the 409 from the database ends up visible in the UI), then engineering practices (tests, one-command run, commits, README). A beautiful UI over an unsafe backend fails this test; a solid backend with rough-but-honest UI can pass. Full rubric is confidential.